

GPU-Initiated Communication: Dissecting Down to the Bone

Javid Baydamirli
Koç University, Türkiye
jbaydamirli21@ku.edu.tr

Ismayil Ismayilov
fal
ismayil@fal.ai

Kaan Oktay
fal
kaan@fal.ai

Didem Unat
Koç University, Türkiye
dunat@ku.edu.tr

Abstract

GPU-initiated communication lets GPU threads post RDMA operations directly to the NIC. It underpins NVSHMEM, NCCL GIN, and DeepEP, which serve the fine-grained, latency-critical communication of Mixture-of-Experts (MoE) models, yet its performance characteristics and optimizations remain scarcely documented beyond source code, and library comparisons fail to separate the costs of the hardware mechanism from those of the library around it.

This paper dissects GPU-initiated communication at the GPU-NIC boundary. We first detail the GPU-side network path: queue placement, work-request construction, doorbell ordering, and completion semantics. We then introduce mini-gda and mini-proxy, minimal transports for the GPU and CPU-proxy submission paths, and measure them alongside NVSHMEM IBGDA, NCCL GIN, DeepEP, UCCL-EP, MSCCL++, and fabric-lib on NVIDIA H100, H200, B200, and GB200 platforms. A minimal GPU path issues an operation in 0.7 μ s and completes in 4.0 μ s; libraries add up to 4.6 μ s of issue time through queue management, memory ordering, and completion scope, and issue time scales with the SM clock. A tuned CPU proxy matches or beats the GPU path at idle, at the cost of a dedicated core whose operating state sets its latency and throughput. On either path, sharing a queue with bulk traffic raises latency by one to three orders of magnitude. Reaching the 260 M msg/s ceiling of our InfiniBand platform requires doorbell batching and queue parallelism, and both have resource costs: communication code can reduce GPU block residency even when unused, and all-to-all traffic loses 59% of its NIC message rate at about 3,000 active connections. The submission path alone therefore does not predict communication performance. Our experiment code and results are available at <https://github.com/ParCoreLab/Dissecting-GPU-Communication-Experiments>.

Keywords

GPU-initiated communication, GPU-initiated networking, RDMA, IBGDA, GDAKI, GPUDirect Async, CPU proxies, NVSHMEM, NCCL GIN, DeepEP, Mixture-of-Experts, expert parallelism, InfiniBand, ConnectX-7, performance characterization

1 Introduction

Communication across GPUs has historically been managed by the CPU, where a host thread builds RDMA work requests, rings the NIC doorbell, and polls for completion. Modern GPU-initiated transports like IBGDA [32] and GDAKI [13] instead move this control to the GPU, enabling GPU threads to construct RDMA descriptors and ring NIC doorbells themselves, over PCIe. Workloads with irregular, latency-critical communication patterns benefit the most from this paradigm. In Mixture-of-Experts (MoE) models, GPU kernels decide which experts each token is sent to, so transfers are fine-grained and data-dependent, and network delay stalls the

GPU directly [24, 53]. This pattern has motivated several specialized communication libraries like DeepEP [53] and pplx-kernels [25] that build on top of GPU-initiated NVSHMEM and NCCL GIN to reduce the latency of dispatch and combine kernels [24, 42]. Conversely, the same workload is also targeted by libraries that opt to keep the host on the communication path. UCCL-EP [31], pplx-garden/fabric-lib [26], and MSCCL++ PortChannel [17] forgo GPU submission to maintain cross-vendor GPU and NIC portability, while still reporting low latency with workload-specific proxy designs.

Published measurements favor each design on a different metric. Proxies report lower latency for a single unloaded operation, and GPU paths higher rates for concurrent small messages [13, 26, 32]. These studies [13, 17, 24, 31, 32, 53], however, typically evaluate libraries as a whole, conflating higher-level software wrappers with the underlying hardware mechanisms. A library-level comparison obscures whether the performance differences come from queue counts, batching strategies, memory fence strengths, or other API overheads, even when said libraries drive the same hardware interface. It may compare an implementation with a single queue against one with sixteen, or a completion over all available queues against one for a single peer.

We therefore isolate each mechanism from the libraries and ask three questions:

- (1) **What is the cost of a single operation?** What does one GPU-initiated RDMA write cost, and how do WQE construction, queue management, memory ordering, completion scope, and the processor clock contribute?
- (2) **When does GPU or proxy submission perform better?** How do proxy design choices such as workers, batching, and queue sharing dictate latency and throughput, and how do the two paths compare under different load conditions?
- (3) **What sustains high message rates, and what does it cost?** How do GPU threads, NIC queues, doorbell batching, and warp and SM placement contribute to throughput, and what do they cost in GPU block residency and NIC connection state?

To answer these questions, we implement two minimal transport drivers: **mini-gda** for the GPU-submission path, and **mini-proxy** for the CPU-proxy path. They perform only the steps the hardware requires and parametrize the design choices, which lets us evaluate each mechanism in isolation without an attached library. Alongside them, we evaluate several production libraries (Table 1) on NVIDIA H100, H200, B200, and GB200 platforms with ConnectX-7 NICs over InfiniBand and RoCEv2.

Table 1: Queue, posting, and completion choices of the implementations we measure. Worker and queue counts are benchmark settings. PE: processing element (one GPU rank); DCI: dynamically connected initiator (§3.4); LL: DeepEP’s low-latency kernels. Versions: NVSHMEM 3.4.5 and 3.7.2, NCCL 2.30.7 and 2.31.2 (Table 2), UCCL-EP a3d520e, MSCCL++ 6231b4f, fabric-lib 2446003.

Implementation	Submitter	Queues and workers	Posting batch	Completion scope
NVSHMEM IBGDA	GPU	RC per peer or a shared DCI pool	default 32; earlier if alone	all configured QPs
DeepEP V1	GPU	LL kernels assign RC QPs to local experts	every 4th message per expert	one (peer, QP)
NCCL GIN GDAKI	GPU	explicit contexts with per-peer RC connections	caller-controlled aggregation	local or context flush
mini-gda	GPU	1–528 QPs, 1–16,384 threads	1–256 WQEs per doorbell	selected QP progress
NVSHMEM IBRC	one CPU proxy	one descriptor FIFO per PE	library-managed	proxy quiet
NCCL GIN Proxy	1–4 CPU workers	per-context descriptor rings	library-managed	context flush
UCCL-EP	1–8 CPU workers	8 FIFOs per worker; MSCCL++ FIFO backend	adaptive chains	channel local
MSCCL++	1–8 CPU services	one FIFO per service, 16 B entries	one request	service local
fabric-lib	one CPU/NIC	host-initiated native client in this study	up to four	transfer local
mini-proxy	1–8 CPU workers	1–32 host rings, 16 B entries	1–16	host or GPU counter

Our contributions are the following:

- A mechanism-level description of the GPU–NIC boundary (§3), covering where GPU-resident RDMA queues live, how device code constructs work requests, and the doorbell, ordering, and completion semantics that libraries implement differently.
- **mini-gda** and **mini-proxy**, two minimal transports that isolate the mechanism from the library, together with a microbenchmark suite built on them (§4). We release both as [open source](#).
- An evaluation across NVIDIA H100, H200, B200, and GB200 platforms (§4) that quantifies single-operation cost, proxy trade-offs, message rate, and the resource costs of the transport, and identifies the configuration choices behind divergent performance across libraries.

The evaluation yields three main findings.

- (1) **Software overhead and completion semantics determine single-operation latency.** A minimal GPU path issues an 8 B write in 0.7 μ s and completes in 4.0 μ s. Communication libraries add varying costs on top through their queue handling, memory ordering, and completion scope: GDAKI takes 1.6 μ s to issue and NVSHMEM’s public API 5.3 μ s. Ordering scope alone changes issue time 3.7 $\times$ across safe configurations, and completing all configured queues doubles put+completion latency from 1 to 16 queues. Issue latency scales strongly with the SM clock.
- (2) **GPU path vs. CPU proxy depends on latency, capacity, and isolation needs.** A tuned proxy completes within 0.07 μ s of the minimal GPU path and has a lower median round trip (5.9 vs. 6.9 μ s), at the cost of a dedicated core whose operating state determines its results. Shared queues degrade under load for both submission paths: a proxy FIFO shared with bulk traffic reaches tens of milliseconds, and a reserved queue recovers one to two orders of magnitude. More workers and batching raise the proxy message rate, which stays well below IBGDA’s on our InfiniBand platform but reaches 90% of it on GB200. Every path except a cold IBRC proxy exceeds 90% of line rate by 7 KiB, the size of a DeepSeek-V3 token’s FP8 hidden state.

- (3) **High message rates require queue scaling and doorbell batching, but both cost resources.** One submitting thread posts 1.8 M msg/s. Cooperative publication lets a warp reach 21.6 M msg/s on one queue, and independent queues raise GPU submission throughput to 260 M msg/s. GPU submission is not required for this peak: a host thread ringing doorbells for GPU-built WQEs reaches 257–258 M msg/s. Communication code can cost up to 37% of a kernel’s useful throughput even when dormant, by lowering block residency. At the NIC, active connections cost more when sending and receiving together: a send-only NIC keeps about 242 M msg/s through 32,768 active queues, while all-to-all traffic loses 59% of its rate near 3,000 connections, and Dynamically Connected (DC) transport does not remove this decline.

Section 2 introduces GPU-initiated communication and the libraries we measure, Section 3 describes the GPU–NIC boundary, Section 4 presents the evaluation, Section 5 discusses related work, and Section 6 concludes with guidelines for library designers and benchmarkers.

2 Background

GPUDirect. GPUDirect is a family of technologies that progressively removes the host from the networking path. GPUDirect RDMA [46] exposes GPU memory over PCIe for direct NIC access, removing host-side staging copies. GPUDirect Async [1] shifts synchronization onto the device, letting GPUs trigger and wait on communication pre-posted by the CPU. Its kernel-initiated successors, IBGDA [34] and GDAKI [13], move submission entirely into the kernel, enabling GPU threads to build queue elements, ring doorbells over PCIe, and poll completions natively.

Two auxiliary components are important to note: GDRCopy [39], which gives the CPU low-latency load/store access to GPU memory and underpins CPU-assisted proxy paths, and `rdma-core`’s `mlx5dv` direct verbs [29], with which user space creates the queues, doorbell pages, and memory registrations that transport libraries place in GPU memory.

RDMA Communication. RDMA communication is organized around Queue Pairs (QPs), each consisting of a Send Queue (SQ) and a Receive Queue (RQ), both structured as circular rings of Work Queue Elements (WQEs). Send and receive completions are directed

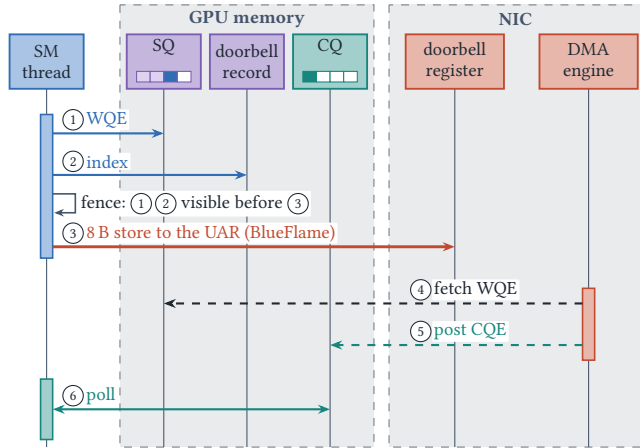


Figure 1: The GPU-submitted path with GPU-resident queues and in-kernel WQE construction. In a CPU-proxy path the host performs (1)–(3) and (6), and the GPU instead enqueues a request descriptor.

to completion queues (CQs), which can be distinct or shared with other QPs. The NIC reports a finished *signaled* WQE by writing a completion queue entry (CQE), while unsigaled WQEs produce no CQE of their own.

Communication Control vs. Submission. We distinguish the agent that *semantically decides* what to communicate from the agent that *physically submits* work to the network [51]. The operation is initiated either by a CPU call (host-initiated), as in classic MPI/NCCL collectives, or by device code (GPU-initiated). A GPU-initiated operation may then be *GPU-submitted*, where the GPU writes the doorbell itself, or *proxy-submitted*, where the request is forwarded to a CPU proxy thread. Additionally, WQEs can be constructed either by the CPU or the GPU, irrespective of the submitter: some implementations delegate only the doorbell submission to the CPU to bypass platform limitations (§3.1).

Table 1 summarizes the queue, posting, and completion choices of the GPU-submitted and proxy-submitted libraries we use in our evaluation.

3 The GPU–NIC Boundary

A network transfer requires NIC queues the submitter can reach, WQE construction, ordered doorbell updates, and completion polling (Figure 1), which we explain in turn. Libraries implement each step differently, and we measure the respective costs of each choice in Section 4.

3.1 Memory geography

In traditional CPU-side RDMA, the SQ and RQ WQE buffers, the completion queue, and the doorbell record typically live in host memory. As GPU submission requires device access to these RDMA queues, implementations like IBGDA and GDAKI allocate them in GPU-accessible memory (typically with `cudaMalloc`), where SMs populate and modify them with loads and stores. The NIC reaches the buffers through GPUDirect RDMA mappings established at registration time with `nvidia-peermem` or DMA-BUF [35, 44].

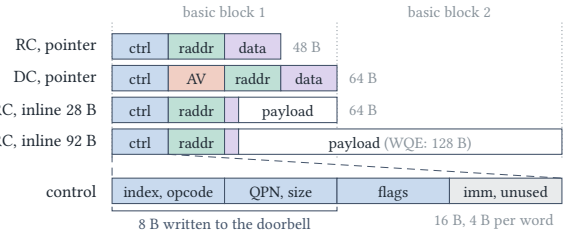


Figure 2: m1x5 send WQEs for an RDMA write, built from 16 B segments (ctrl: control, AV: address vector, raddr: remote address) and fetched in 64 B basic blocks. Only DC WQEs carry the AV (§3.4). An inline payload replaces the data segment’s pointer with a length word and the payload. The bottom row expands the control segment; the doorbell store carries its first 8 B.

While the queues can live in any memory subsystem the NIC can reach by DMA, the doorbell register cannot be freely moved. It is a hardware register inside a *User Access Region* (UAR), which is a slice of the NIC’s PCIe BAR through which user-space processes submit doorbells. The UAR must be mapped into the GPU’s virtual address space for kernels to submit doorbells directly. This is achieved by allocating the UAR with `m1x5devx_alloc_uar`, and registering its BAR page with the CUDA driver as I/O memory (`cuMemHostRegister`), which can then be used as an opaque device pointer that SMs can store to (`cuMemHostGetDevicePointer`) [44].

Mapping a third-party PCIe BAR into GPU address space is a potential security hazard, and the NVIDIA kernel module permits it only with the option `PeerMappingOverride=1` [23]. DMA-BUF registration lets the NIC access GPU memory but does not by itself establish this reverse mapping [44]. NVSHMEM’s *CPU-assisted* mode keeps GPU-built WQEs and has a host thread forward the doorbell value to the UAR for systems where the mapping is disallowed [23].

3.2 Work requests, submission, and doorbell semantics

WQE anatomy. An m1x5 send WQE is a concatenation of 16-byte segments packed into 64-byte basic blocks, shown in Figure 2 for an RDMA write. The payload can either be named by the data segment or packed directly into the WQE, trading the NIC’s DMA read of the payload for a larger WQE (§4.1). DC transports additionally include an address-vector segment to identify the dynamically connected target (DCT).

Submission. Doorbell posting is a two-step process. First, the submitting thread updates the doorbell record (dbrec) with the index of the next free WQE basic block, which the NIC reads to get the number of posted WQEs [33]. The thread then performs an ordered 64-bit store to the doorbell register on the UAR to notify the NIC of available work. The doorbell store carries the first 8 bytes of the WQE’s control segment, which pack the WQE index and the QP number.

The doorbell targets the UAR’s send-doorbell/BlueFlame region. WQEs can either be copied into the BlueFlame buffer via write-combining stores, or DMA-fetched by the NIC separately. Inline copying inflates the MMIO write payload of the doorbell, whereas NIC fetching preserves a fixed doorbell size regardless of message

length. Every GPU-side implementation we evaluate opts for fixed-size notifications and NIC-fetched WQEs [10, 37, 43], whereas host-side rdma-core inline-copies small WQEs into the BlueFlame buffer [28].

Doorbell Batching. Announcing several WQEs with one doorbell raises throughput, with a possible latency penalty if submitters wait to form batches. The mechanism has been studied for CPU-side RDMA [20], and GPU libraries make use of the same optimization. NVSHMEM defers the doorbell until a configured number of WQEs is ready (default 32), but posts isolated operations immediately [43, 45]. NCCL GDAKI exposes caller-controlled aggregation through its device API and DOCA posting path [37, 41], and DeepEP’s low-latency kernels ring on every fourth message of an expert’s queue [10]. The batch a library achieves therefore depends on how many threads post at once and in what order they publish, not only on its threshold. We measure the effects of batching in Section 4.3.1.

Sharing a QP. Threads can share a QP by reserving consecutive WQE slots, building their WQEs in parallel, and publishing them in reservation order before one doorbell announces them. Publication is serialized: a thread cannot publish past an unfinished predecessor, so lanes that drift apart wait for each other. Cooperative publication avoids this by letting a warp reserve, build, and publish its WQEs together, paying the reservation and the doorbell once per warp [37, 43]. Section 4.3.1 compares these strategies.

Ordering. Correct GPU-side RDMA delivery hinges on two ordering requirements: (1) WQE and source-payload stores must be visible to the NIC before the doorbell that announces them; and (2) when one thread rings the doorbell for WQEs that other threads wrote, each writer’s stores must be visible before the ringing thread sees its slot as ready. NVSHMEM implements both with plain stores by default, each preceded by `__threadfence()` (`__threadfence_system()` when the queues reside in host memory). When built without host-side queue support¹, it instead uses GPU-scope release stores² on the doorbell path. NCCL’s bundled DOCA implementation supports a GPU-scope release fence followed by a relaxed MMIO store, along with architecture and scope-dependent alternatives [37]. We measure the cost of each of these orderings in Section 4.1.

3.3 The completion path

Upon executing a signaled WQE, the NIC writes a CQE that identifies the completed work and reports its status. Because send queues are strictly ordered, a single CQE implicitly retires all preceding unsigned WQEs on the same queue [27, 28]. GPU threads poll the CQ ring to consume these entries, but completion scope varies across libraries. NVSHMEM’s quiet walks all configured RC QPs and the DCI pool, while DeepEP V1 polls a selected (peer, QP), usually one QP per local expert [10, 43]. NCCL GIN’s `flushAsync` and `wait` complete one peer on one context, while `flush` covers the context’s pending transfers for the participating threads [40]. We measure the cost of the completion scope in Section 4.1.

The NIC writes both remote payloads and local CQEs into GPU memory by DMA, and an SM accessing them observes them within the GPU memory hierarchy. Under its relaxed memory model, a thread that observes a given CQE — or any arbitrary flag in memory — is not thereby guaranteed to observe the payload written before it. Libraries handle this in their wait and signal primitives, and user code that polls raw memory must also handle it for correctness. This is a common hazard for persistent kernels, which lack the implicit synchronization of a kernel-launch boundary [14, 19, 49, 51], and for barrier-free NVLink collectives, which avoid it by writing the flag and the data in one atomic store [47].

3.4 Transport choice

The transport decision influences both communication latency and scalability. Reliable Connection (RC) transport binds each local QP to one remote QP at creation time. Pre-establishing this connection eliminates the need to specify destination addresses in individual send WQEs, but forces each peer to establish a dedicated QP for every other remote endpoint. With parallel submitters needed to saturate the NIC (§4.3.1), RC QPs may be allocated per GPU, CTA, or warp, growing the local QP state. The associated QP contexts, CQs, WQEs, and memory translations occupy NIC caches and host-backed state, and poor locality across them reduces message rate [21]. Section 4.3.3 measures this for GPU-driven traffic.

Dynamically Connected (DC) transport, by contrast, lets a pool of dynamically connected initiators (DCIs) address many remote DC targets (DCTs). This reduces the number of initiator QPs at the cost of an additional WQE segment in each send. Each DC send WQE carries an address vector of 16 B (48 B on HCAs without the compact form [29]) identifying its destination. Changing a DCI’s destination may incur additional work on the NIC [38, 43]. Section 4.3.3 measures both costs. The same per-peer state pressure has motivated transport redesigns beyond InfiniBand: the Ultra Ethernet Transport replaces the long-lived per-peer connection state of RC with packet-delivery contexts that are established and released on demand [50].

4 Mechanism Evaluation

We organize the experiments and evaluations around the three questions introduced in Section 1.

Minimal Implementations. We implement two minimal paths to accompany the production libraries listed in Table 1. **mini-gda** is a GPU-initiated communication implementation with GPU-resident NIC queues and the doorbell register mapped into GPU address space. GPU threads construct WQEs, update queue state, ring the doorbell, and poll completions, without a surrounding library. Ordering is selected at build time, and payload placement, queue count, and doorbell batching are launch parameters. **mini-proxy** is a proxy implementation where GPU threads write 16 B request descriptors into rings in host memory, from which CPU workers construct RDMA writes and submit them through `ibverbs`. Worker count, ring count, and the number of requests submitted together are configurable. We measure DeepEP V1, whose device path uses NVSHMEM. DeepEP V2 replaces the NVSHMEM backend with NCCL GIN; since we measure GIN’s GDAKI path directly, the two cover both DeepEP backends at the mechanism level.

¹NVSHMEM_IBGDA_SUPPORT_GPUMEM_ONLY=ON

²PTX st.release.gpu.global.L1::no_allocate

Table 2: Evaluation platforms.

	P-IB	P-RoCE	P-H100	P-GB200
Nodes × GPUs	4 × 4 H200	2 × 8 B200	100 × 4 H100	2 × 1 GB200
CPU	Xeon 6548Y+	Xeon 8581C	Xeon 8460Y+	Grace
NICs / node	4× CX-7	8× CX-7	4× CX-7	1× CX-7
Link	200 Gb/s IB	400 Gb/s RoCEv2	200 Gb/s IB	400 Gb/s IB
Driver / CUDA	580.95 / 13.0	590.48 / 13.2	595.71 / 12.6	580.159 / 13.0
NVSHMEM	3.4.5 / 3.7.2	3.7.2	3.4.5 / 3.7.2	3.7.2
NCCL	2.30.7 / 2.31.2	2.31.2	—	2.31.2
Doorbell writer	GPU	GPU	host thread	GPU

Platforms. Table 2 shows the platforms used in our evaluation. **P-IB** is the primary platform for GPU-submitted networking and the proxy comparison. **P-RoCE** is used for the SM-clock and completion-scope experiments, as well as cross-platform controls. **P-H100** is a larger cluster without NIC doorbell mapping support used for active-connection experiments. **P-GB200** is a Grace–Blackwell system with coherent memory between the CPU and the GPU, where host rings and counters used by proxies travel over NVLink–C2C rather than PCIe. We repeat the one-operation and proxy experiments on P-GB200 with one GPU and one NIC per node. We use 8 B operations to expose per-operation control costs, then increase payload size to identify when link bandwidth dominates.

Timing and Validation. We time in-kernel operations with `globaltimer` and use `clock64` to record the effective SM clock. End-to-end measurements use CUDA events. We validate payloads after each run and, where applicable, check NIC counters against the expected traffic, including protocol overhead (Appendix A).

Proxy Clock State. On P-IB, CPU-proxy latency and rate depend on host operating state. We compare *cold* workers after 45 s idle with *warm* workers after 20 s of sustained load; telemetry shows base and turbo clocks, respectively. P-IB proxy tables report cold values unless marked; Appendix C.1 gives both states.

Pairs and Placement. Unless stated otherwise, latencies are p50 over three independent process runs on one pinned pair of nodes, using the NIC attached to the GPU’s PCIe switch.

4.1 The cost of one operation

Question. What does one GPU-initiated RDMA write cost the issuing thread, how much of that is the mechanism and how much the library, and how does handing the operation to a CPU proxy compare?

Setup. One GPU thread issues 8 B puts to a remote processing element (PE), with at most one outstanding put. We time the following:

- **Issue:** Time the GPU thread spends submitting the put, including WQE construction, queue management, ordering, and doorbell stores. Completion time is excluded.
- **Put+completion:** Issue and completion-routine return, including polling, ordering, and queue updates.
- **Round trip:** Time to send a request and observe the remote GPU’s reply.

Table 3: One 8 B operation. Completion is one CQE for mini-gda, a host-resident counter for the baseline mini-proxy, and a GPU-resident counter written through GDRCopy for the tuned one.

Path	P-IB (μs)			P-GB200 (μs)		
	Issue	Put+c.	RTT	Issue	Put+c.	RTT
GPU submits						
mini-gda, inline	0.70	4.03	6.85	0.77	5.70	8.64
mini-gda, non-inline	0.70	4.64	—	0.80	6.62	—
mini-gda, unordered (unsafe)	0.19	3.46	—	0.16	5.09	—
GDAKI, inline	1.60	6.02	10.50	1.60	7.26	14.40
GDAKI, non-inline	1.82	6.85	—	1.79	8.48	—
NVSHMEM internal	4.26	9.18	—	5.50	12.19	—
NVSHMEM public	5.31	11.10	21.50	6.62	13.82	25.25
CPU proxy submits						
NVSHMEM IBRC	0.99	6.40	11.07	1.18	7.14	12.42
NCCL GIN Proxy	2.46	8.19	16.32	2.27	7.87	17.54
mini-proxy, baseline	2.27	7.07	14.59	2.24	6.91	14.91
mini-proxy, tuned	0.13	4.10	5.89	0.13	4.99	7.46

We compare three GPU-submitted stacks:

- **mini-gda:** One RC QP, GPU-scope ordering as in DOCA, and completion by polling its CQ.
- **NCCL GIN GDAKI:** One context and one RC QP per peer, completing with `flushAsync` and `wait` without doorbell aggregation.
- **NVSHMEM:** Two RC QPs per peer (default) through the internal `nvshmemi_ibgda_rma_nbi` + internal quiet, and public `nvshmem_putmem_nbi` + `nvshmem_quiet`. We separately measure 16 QPs, which achieves the highest message rate (§4.3.1).

In addition to the GPU-submitted paths, we report NVSHMEM IBRC, NCCL GIN Proxy, and mini-proxy as CPU-submitted references. A proxy’s issue interval measures GPU enqueue without the host-side WQE construction. Round trips use each stack’s own notification mechanism: put-with-signal for NVSHMEM, putValue with a signal increment for NCCL, and a polled data word for mini-gda and mini-proxy.

The mechanism costs 0.7 μs to issue. Constructing the WQE, updating the doorbell record, and writing the doorbell to the NIC takes mini-gda 0.70 μs (Table 3). GDAKI is the next fastest GPU-submitted path, and NVSHMEM’s internal and public paths add significant further costs before completion (Table 3). Replacing NVSHMEM’s default fence with a GPU-scope release recovers only 0.16 μs of issue on the same pair. NVSHMEM’s additional work includes slot reservations, ready-head updates, doorbell locks, and QP lookups [37, 43]. The SM-clock experiment later in this section tests how much of the issue gap scales with device execution speed. P-GB200 repeats the ranking of the GPU-submitted paths (Table 3).

Ordering changes the latency floor. WQE stores must become visible before the doorbell announces them for the NIC to read the correct data (§3.2). Table 4 compares different ordering configurations used by production libraries in mini-gda. GPU-scope release ordering has a lower cost than system scope on both platforms: a system-scope fence costs 3.7× the issue time of a GPU-scope fence on P-IB (2.62 vs. 0.70 μs). Shen et al. report the same pattern over NVLink, where one barrier costs more than 1 μs against a

Table 4: mini-gda ordering configurations. Each row orders the WQE and doorbell-record stores before the doorbell store (§3.2) and polls the CQ at the matching scope.

Ordering	P-IB (μ s)		P-GB200 (μ s)	
	Issue	Put+c.	Issue	Put+c.
None (unsafe control)	0.19	3.46	0.16	5.09
GPU-scope fence (DOCA, GDAKI)	0.70	4.03	0.77	5.70
+ fenced doorbell record	0.93	4.26	0.99	5.89
GPU-scope release store (DeepEP)	0.70	4.00	0.77	5.66
__threadfence() (NVSHMEM)	1.22	4.48	1.22	6.14
System-scope release store	2.37	5.66	2.08	7.01
System-scope fence	2.62	5.92	2.34	7.26

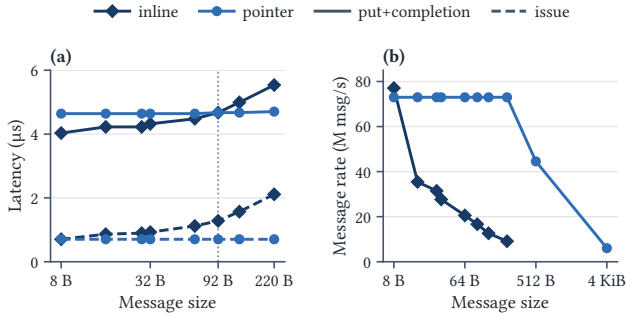


Figure 3: Inline vs. pointer payloads in mini-gda on P-IB. (a) One-thread issue and put+completion latency against inline payload size. (b) Message rate at 16 QPs with 16 WQEs per doorbell.

1.4 μ s data-movement floor [47]. Omitting fences and using plain CQ loads achieves the lowest latencies — 0.19 μ s issue and 3.46 μ s through completion — but is not guaranteed to be safe (§3.2). We found no corruption with the ordering removed. While this does not prove it safe in general, an implementation could drop the fence for a workload it has verified to get closer to the mechanism’s floor.

Completion arrives 3 μ s after the doorbell. An instrumented run observes a valid CQE 3.30 μ s after the doorbell store, covering doorbell delivery, NIC and network processing, and GPU polling. The rest of each path’s put+completion time in Table 3 is spent in its completion routine, which bundles polling, ordering, and queue updates, so we do not attribute it to one mechanism. Every GPU-submitted path completes 1.2–3.0 μ s later on P-GB200 than on P-IB, while issue times stay within 1.3 μ s. On P-GB200 the NIC reaches GPU memory through the Grace CPU and NVLink-C2C rather than through a shared PCIe switch, so every WQE fetch, payload read, and CQE write takes a longer path. Consistent with this, moving mini-gda’s queues to host memory there shortens put+completion by 0.86 μ s at the same ordering.

Inlining trades a source read for GPU stores. Inlining an 8 B value spares the NIC a read of the source buffer at no extra issue cost, saving 0.61 μ s of completion time. At larger inline payloads, however, the issue cost increases, and the batched message rate falls (Figure 3). We find that issue time grows by about 0.1 μ s per additional 16 B chunk, and the completion savings are gone by 92 B. NVSHMEM, GDAKI, and DeepEP inline only 8 B values.

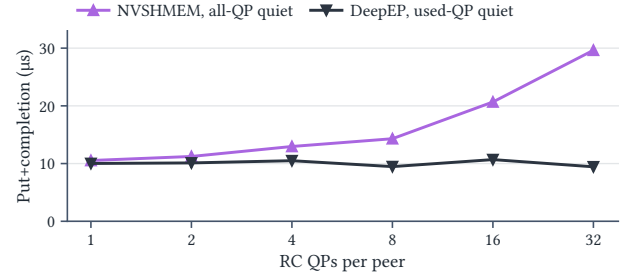


Figure 4: Completion scope against queue count on P-IB. Both quiet curves use the same internal put (NVSHMEM 3.4.5); the all-QP quiet sweeps every configured RC QP, while DeepEP’s ported quiet polls only the used QP.

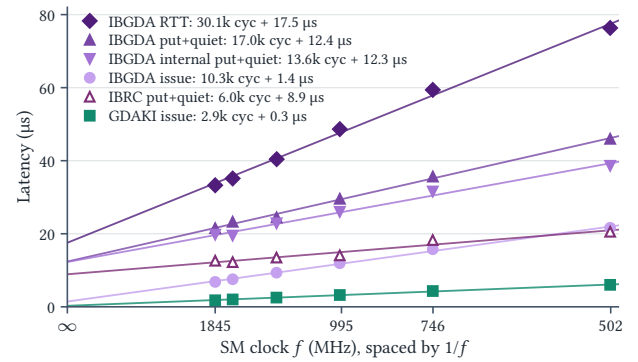


Figure 5: Latency against $1/f$, where f is the locked SM clock. Both GPUs are on the same node, each with its own NIC across the fabric. The slope is an effective clock-sensitive coefficient in cycles and the intercept is the fitted clock-insensitive component.

All-QP quiet grows with queue count. While adding queues increases posting capacity, they each individually require completions. NVSHMEM’s PE-wide quiet visits all configured QPs regardless of use, so its put+completion latency doubles between 1 and 16 QPs (Figure 4), while DeepEP’s ported single-QP quiet stays flat, as does a signal-based round trip. NVSHMEM’s own per-QP quiet is within 0.5 μ s of the port at 16 QPs, so the gain comes from narrowing the completion scope rather than faster queue handling. NVSHMEM 3.7.2 on P-RoCE shows the same growth to 32 QPs (Appendix Table 7). Letting NVSHMEM size its DCI pool automatically adds a further 81 μ s on P-GB200 without changing issue time (Appendix Table 8).

A throttled SM is a slower communicator. On P-RoCE, lowering the SM clock slows issue more than put+completion for both NVSHMEM and GDAKI (Figure 5). At the top clock, the clock-sensitive term is 80% of NVSHMEM’s public issue and 85% of GDAKI’s, but NVSHMEM’s coefficient is 3.5 $\times$ larger, which accounts for most of the issue-latency gap. A throttled GPU can therefore communicate more slowly over the same network. Offloading to a CPU proxy does not remove this dependence, as the GPU still enqueues the request: the fit attributes 92% of IBRC’s enqueue time and 27% of its put+completion to the SM clock (Appendix Table 6).

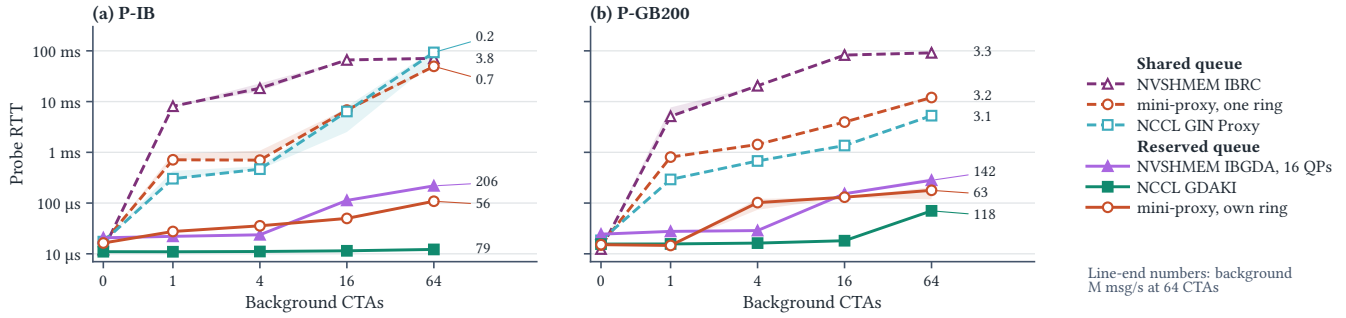


Figure 6: RTT under load on (a) P-IB and (b) P-GB200, with background CTAs loading both endpoints. Dashed curves share one ring or context with the bulk traffic; solid curves reserve a probe queue. IBGDA uses a fixed 16-QP pool and GDAKI a private context. Labels give the background M msg/s carried at 64 CTAs.

Reducing the Proxy Enqueue Cost. mini-proxy’s baseline protocol takes $2.27\ \mu\text{s}$ to enqueue a request, including a device atomic for ring reservation, a PCIe read of host-resident progress, and a system-scope release. We can tune the proxy to the workload by allocating one producer per ring, caching progress, and encoding validity in the descriptor, which together reduce enqueue to a single posted 16 B store taking $0.13\ \mu\text{s}$. The tuned proxy matches mini-gda through completion on P-IB and has a lower median round trip on both platforms (Table 3). These results require a dedicated CPU worker pinned to a core on the NIC’s NUMA node (§4.2).

Proxy latency depends on host operating state. On P-IB, CPU-only warm-up improves proxy latency and tails while leaving GPU-submitted paths largely unchanged. When warm, the tuned proxy’s round-trip p99 falls from 8.9 to $5.6\ \mu\text{s}$, and IBRC’s small-message rate doubles from 1.9 to $3.8\ \text{M msg/s}$ (Appendix C.1). This sensitivity shows that worker placement and host-side preconditioning must be considered for reproducible proxy comparisons.

Takeaway. Software choices, not the hardware mechanism, set single-operation latency. A minimal GPU path issues an $8\ \text{B}$ write in $0.7\ \mu\text{s}$ and completes in $4.0\ \mu\text{s}$, while libraries add up to $4.6\ \mu\text{s}$ of issue time through queue management, ordering, and completion scope. Ordering scope alone changes issue time $3.7\times$ across safe configurations, and all-QP completion doubles put+completion from 1 to 16 QPs while per-QP completion stays flat. Issue time follows the SM clock on both paths, including a proxy’s GPU-side enqueue. A tuned proxy matches the minimal GPU path through completion and has a lower median round trip, at the cost of a dedicated core whose clock state sets its latency, tail, and rate.

4.2 The proxy design space

Question. Several libraries argue that a well-designed proxy matches GPU submission [17, 26, 31], while others [13, 32, 53] report latency and message-rate wins for GPU submission. The libraries make different choices for threads, rings, batching, and the GPU-CPU handoff (Table 1). When can a CPU proxy match GPU submission, and which of these choices matter where?

Setup. Four design choices differ across proxy implementations (Table 1): the descriptor rings (R); the CPU workers (T) that poll rings and post requests, each with its own NIC QP; the batch of work requests (B) chained into one send (`ibv_post_send`); and the GPU-CPU handoff, that is, how GPU threads reserve slots, publish descriptors, and observe worker progress.

We measure the round trip of an $8\ \text{B}$ request-reply with and without background CTAs issuing writes through the same NIC from another stream. We also report the message rate and goodput when many GPU threads issue puts before waiting for completion. Both endpoints generate background traffic. We report the rate carried during the probe alongside latency, since each configuration sustains a different total load.

Proxy workers are placed near the GPU’s NIC: mini-proxy and UCCL-EP pin workers to cores, while MSCCL++ binds them to the local NUMA node. The loaded, message-rate, and payload experiments are repeated on P-GB200 for mini-proxy, IBRC, GIN Proxy, IBGDA, and GDAKI. fabric-lib is host-initiated by design [26], so we drive its native client from a host loop and time it on the CPU; its GPU-to-host handoff is not measured.

Latency at Idle. Published evaluations place proxies and GPU-submitted transports in overlapping ranges, and Hamidouche et al. measure IBRC’s round trip below IBGDA’s [13]. We reproduce this with IBRC answering a round trip in $11.1\ \mu\text{s}$ against NVSHMEM’s public 21.5 (Table 3). These results do not establish an intrinsic proxy advantage, as the minimal GPU path and GDAKI both answer faster than IBRC. The other proxies’ round trips reflect their notification protocols: fabric-lib answers in $10.1\ \mu\text{s}$ with its write-immediate counter, UCCL-EP in 13.1 polling the received word, and MSCCL++ in 18.5 with a semaphore and per-iteration flush.

Shared queues suffer under load. The latency measurements shift when background traffic is introduced (Figure 6). Every proxy that sends the probe through a queue shared with bulk traffic loses three orders of magnitude: IBRC’s single FIFO, mini-proxy with one ring, and GIN Proxy’s shared context all reach tens of milliseconds at 64 background CTAs while carrying only $2\text{--}4\ \text{M msg/s}$. IBRC, which reported competitive round-trip latency at idle, collapses $670\times$ with a single background CTA. These configurations force latency-sensitive requests through the same FIFO as bulk traffic

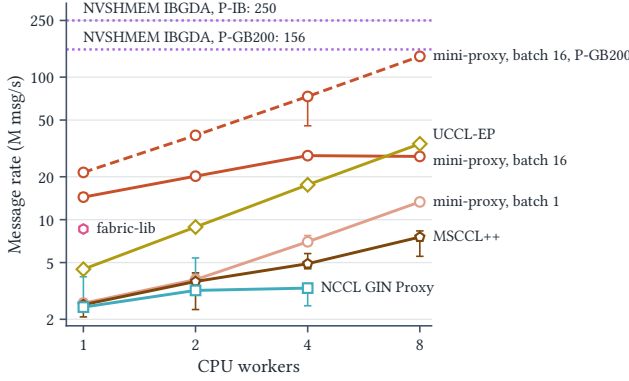


Figure 7: 8 B proxy message rate against worker count on P-IB. Batch counts the requests mini-proxy chains into one post; batch 16 is repeated on P-GB200 (dashed). fabric-lib runs one worker per NIC. Dotted lines mark NVSHMEM IBGDA on the same pairs. Worker cores are at their base clock; turbo raises every proxy rate 1.3–3.2× (Appendix C.1).

with no isolation or priority. GPU submission does not isolate by itself either: a single GDAKI context shared by the probe and all background CTAs answers in 362 μ s at 64 CTAs while carrying 2.6 M msg/s, whereas a private context keeps p50 at 12–16 μ s with the background rate at 73–79 M msg/s.

Queue reservation reduces interference. Reserving a queue for the probe recovers most of the loss: UCCL-EP and MSCCL++ go from milliseconds to 120 and 825 μ s. With workers, rings, and batching held fixed, a private mini-proxy ring cuts the loaded round trip by one to two orders of magnitude on both platforms, and a private UCCL-EP FIFO or a reserved MSCCL++ service does the same (Figure 6, Appendix C.2). GPU submission exhibits the same behavior: with GDAKI’s 65-context pool and carried load held fixed, moving the probe off a context shared with a single background CTA halves its latency, from 27.3 to 13.9 μ s (Appendix Table 11). Reserving a worker as well adds little and takes capacity from bulk traffic, but shared workers and NIC resources still permit interference, so latency must also be read with the load carried. Paced to the same 30 M msg/s, a reserved GDAKI context still answers in less than half the time of a reserved mini-proxy ring, and comparing them at their unrestricted loads would mix isolation with posting capacity.

Message rate: chaining and workers raise capacity. As with GPU submission, proxy message rate can be raised by batching (chaining) and parallel submitters. GPU threads construct WQEs in parallel, whereas each CPU worker serializes its posting work. Chaining requests amortizes that work, and additional workers provide parallel submission (Table 1). Libraries implement a mix of the two: MSCCL++³ adds workers as services but posts each write individually, fabric-lib chains up to four requests on one worker, UCCL-EP drains several FIFOs into variable-length chains, and mini-proxy exposes T , R , and B separately (Appendix Table 10). Figure 7 shows the resulting scaling. At $T = 1$, $R = 32$, increasing

³We patched a shared-pointer refcount bug that serializes MSCCL++’s services (Appendix C.1); we use this build throughout and retain the one-request posting policy.

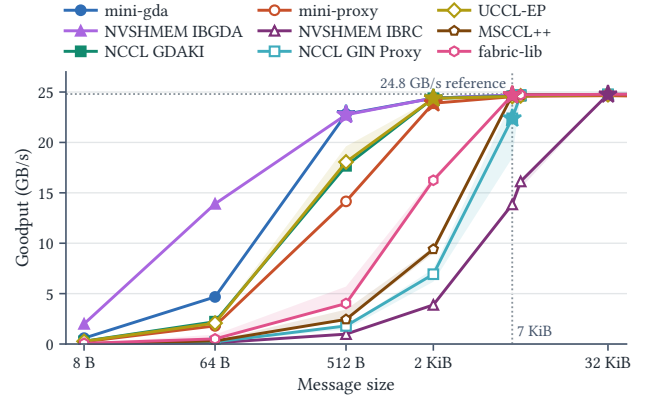


Figure 8: Pipelined goodput against payload size on P-IB. Stars mark the first sampled size at 90% of the 24.8 GB/s reference. Vertical line is a DeepSeek-V3 FP8 token vector (7,168 B). Proxies use four workers (UCCL-EP eight, fabric-lib and IBRC one). IBRC is measured cold and the other proxies’ clock state was not controlled. Producer CTAs fall from 16 to one with size.

mini-proxy’s batch limit B from 1 to 16 raises the message rate 5.5×. Adding workers scales every library tested up to a point, though they still stay an order of magnitude below NVSHMEM IBGDA’s 250 M msg/s (§4.3.1). Under load, the batch costs little latency: with one worker, eight rings, and 64 background CTAs, a private probe answers in 272 μ s at $B = 16$ against 255 μ s at $B = 1$ while the background carries 5.2× more traffic. Independent workers still need to avoid contending on shared state: GIN Proxy stays near 3 M msg/s over one to four workers in the tested four-context configuration, and MSCCL++’s services scale only with the refcount patch. fabric-lib’s single worker reaches 8.6 M msg/s by amortizing its native calls across pagged chains of up to four requests.

The proxy’s ceiling is platform-dependent. On P-GB200, mini-proxy continues scaling to eight workers at $B = 16$, reaching 140 M msg/s, or 90% of IBGDA’s 156 M msg/s there, against 250 on P-IB (Figure 7), although CPU, link, firmware, and RDMA provider all differ between the pairs. Coherent NVLink-C2C helps capacity but not latency: the tuned proxy’s round trip is 7.5 μ s on P-GB200 against 5.9 μ s on P-IB (Table 3).

Bandwidth: larger payloads hide the posting-rate differences. Posting rate sets the payload size needed to fill the link (Figure 8). The experiment compares three GPU-submitted paths with six proxy implementations on one pair. The first measured size reaching 90% of 24.8 GB/s is 512 B for NVSHMEM IBGDA and mini-gda. GDAKI, mini-proxy, and UCCL-EP reach it at 2 KiB, while MSCCL++, GIN Proxy, and fabric-lib lag until 7 KiB. Paths with higher small-message rates saturate the link with smaller payloads. At 7,168 B, the size of a DeepSeek-V3 token’s FP8 hidden-state vector [9], all paths in the sweep except cold IBRC exceed 90% of the reference bandwidth; IBRC reaches it at 32 KiB cold and 7 KiB warm. On P-GB200’s 400 Gb/s link, mini-proxy, IBGDA, and GIN Proxy also approach line rate at 4 KiB (Appendix Table 12). Completion latency remains different even as goodput converges (Appendix C.3).

Table 5: mini-gda 8 B message rate on P-IB. Per QP is the total rate divided by the QP count. ‡: one QP per CTA shared cooperatively, reserved per warp and rung once per 32 WQEs, as NVSHMEM does. †: one doorbell lock per WQE. *: slots handed off per thread rather than per warp.

	Thr.	QPs	SMs	Batch	M msg/s	per QP
One thread	1	1	1	1	1.79	1.79
	1	2-32	1	1	1.85	—
	1	1	1	16	4.70	4.70
One QP	32 [‡]	1	1	1	1.27	1.27
	256*	1	1	32	2.60	2.60
	32 [‡]	1	1	32	21.6	21.6
	256 [‡]	1	1	32	30.7	30.7
Several QPs	16	16	16	1	27.9	1.74
	16	16	16	16	74.4	4.65
	32	32	1	16	133	4.15
	64	64	1	16	206	3.21
	64	64	64	16	260	4.06
NVSHMEM v3.7.2	256	1	1	32	25.2	25.2
	4,096	16	16	32	250	15.6

Takeaway. Idle latency reflects per-operation software costs, while loaded latency depends on queue isolation and carried traffic. Separate rings reduce interference, while workers and chaining raise proxy capacity. Larger payloads hide posting-rate differences in pipelined goodput, but completion latency remains a separate consideration. Proxy capacity stays well below GPU submission on P-IB but reaches 90% of IBGDA’s rate on P-GB200.

4.3 Message rate and resource costs

4.3.1 What does message rate take? A single submitting thread cannot saturate the NIC, and concurrent submission is needed. How does message rate grow with threads, CTAs, and QPs, and what does doorbell batching add?

Setup. On P-IB, GPU threads issue 8 B inline puts to one remote GPU with many writes outstanding, periodic queue reclamation, and a final drain. mini-gda varies threads, QPs, SMs, and WQEs announced per doorbell (Table 5). NVSHMEM runs with 16 RC QPs and 256 threads per CTA, and GDAKI with one context and eight threads per CTA, each library’s best measured configuration.

Cooperative sharing raises the rate of one queue. One thread with one QP and one WQE per doorbell posts 1.79 M msg/s (Table 5). Adding parallelism to either the QPs or the threads individually does not meaningfully change the posting rate. Sharing the QP cooperatively instead raises the rate: a warp can reserve 32 slots and build its WQEs in lockstep, with one lane publishing the group with a single ring (as NVSHMEM does). The same QP in this regime posts 21.6 M msg/s from one warp and 30.7 from two or more warps, because the reservation, the ordered doorbell sequence, and the QP lock are paid once per warp rather than once per WQE. Adding more threads or larger doorbell batches does not raise the single-QP ceiling in our experiment.

Batching raises the per-thread rate and queues multiply it. Batching amortizes the ordered doorbell sequence, including its 8 B MMIO store, across several WQEs. Announcing 16 WQEs per doorbell gets one thread a 2.6× boost from 1.79 to 4.70 M msg/s,

with diminishing gains from larger batches. GDAKI’s doorbell aggregation raises its rate by about 6× per context, and NVSHMEM batches ready WQEs across threads. This is the GPU equivalent of host-side doorbell batching [20].

GPU submission is not needed for peak rate. A host thread ringing the doorbells for GPU-built WQEs reaches 257 M msg/s at 16 active QPs on P-H100 and 258 M msg/s on P-IB, at the cost of 2.6–5.8 μs more put+completion.

Publication order limits library rates. The libraries fill their queues differently. NVSHMEM batches ready WQEs across warps; GDAKI aggregates per thread but publishes in reservation order. Lanes that drift apart therefore wait for each other. At 32 threads per context, a __syncwarp after each put roughly triples GDAKI’s rate (Appendix D.1). The minimal path likewise shows that cooperative publication is much faster than handing off individual slots on the same QP. More submitting threads help only when publication can keep up.

Takeaway. Message rate depends on independent queues, batching, and how warps and QPs are shared: mini-gda and NVSHMEM reach the 250–260 M msg/s ceiling with 16 shared QPs and 4,096 threads. mini-gda also reaches this rate with 64 batched single-thread QPs. Increasing NVSHMEM from the default 2 to 16 QPs raises rate about 5× at one CTA per QP. The queues that buy rate are paid for elsewhere: each lengthens a PE-wide quiet (§4.1) and is one more active connection for the NIC (§4.3.3).

4.3.2 What does the enclosing kernel pay? Communication code can change a kernel’s register allocation, block residency, and spilling even when unused. Section 4.1 measured the submitting thread’s latency; what does carrying this code, executing it, and waiting for completion cost the kernel’s useful work?

Setup. On P-IB, compute (FMA) and memory-bound streaming kernels keep N live values per thread across communication calls, in 528 blocks of 256 threads (four per SM on average), so every variant has the same useful work and launch shape. Each block sends 0, 1, 4, or 16 individual 8 B or 7 KiB messages spread through its work. We compare a kernel without communication code to three variants: **code only**, where the path is compiled in but its branch is never taken; **send, then compute**, where one thread per block submits each message and its warp resumes computing while the transfer proceeds; and **send, wait, then compute**, where that warp first waits for the message’s completion. Other warps compute throughout. mini-gda (one QP per block) and mini-proxy (four workers, 32 rings) are the bare paths; NVSHMEM (16 RC QPs per PE) is built both with separately compiled device functions and with its body inlined into the caller, and GDAKI (32 contexts) submits with put plus flushAsync. Times measure kernel execution, excluding the final completion drain (Appendix D.2).

Dormant communication code can reduce throughput. Adding communication increases register use, but its throughput cost depends on the caller. In the compute caller (Figure 9a), mini-gda, mini-proxy, and separately compiled NVSHMEM retain nearly all throughput, while inlined NVSHMEM and GDAKI lose 11–15%. In the streaming caller, both NVSHMEM builds and GDAKI

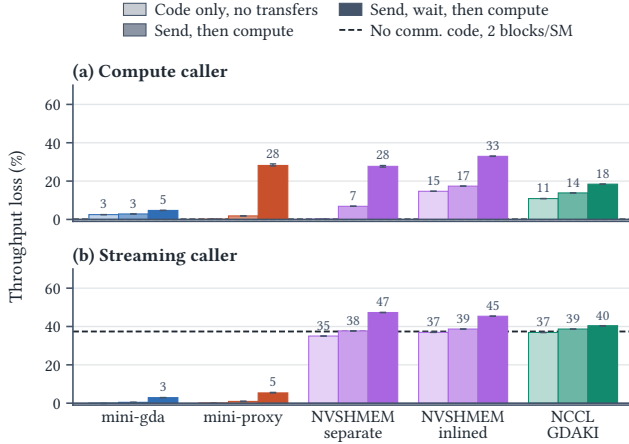


Figure 9: Useful-work throughput loss relative to the kernel without communication code, for (a) a compute caller with eight live values (2.1 ms) and (b) a streaming caller with 16 live values (2.6 ms). The three bars of an arm are not additive.

lose about 37% before sending a message, consistent with lower block residency (Figure 9b).

A separate run with eight blocks per SM shows that even mini-gda’s dormant path costs a streaming caller 27% by lowering residency. Inlining also changes the trade-off: it raises register demand in a thin caller but postpones spilling as the caller’s live state grows, because a separately compiled call forces the caller’s live values to survive it (Appendix D.2).

Waiting costs more than intermittent submission. Sending without waiting adds little loss, since even at 16 messages per block the compute caller averages only about 4 M msg/s. Waiting for each 8 B message reduces throughput by a further 2–5% with mini-gda or GDAKI and 19–27% with NVSHMEM or mini-proxy. Queue count amplifies NVSHMEM’s waiting cost: with 16 live values, separately compiled NVSHMEM’s waiting loss grows from 7% to 20% and 64% at 16, 64, and 528 RC QPs per PE.

In DeepEP, issuing finishes early. In DeepEP V1’s intact low-latency kernels on eight P-IB GPUs, dispatch finishes issuing about 25 μ s into a 0.49 ms operation; the rest is receive-side waiting and copying. Combine also finishes issuing early, then waits at grid synchronization (Appendix D.2). Reducing issue cost alone therefore addresses only a small part of their execution.

Takeaway. A kernel can pay for communication before it sends a message. Library packaging, available overlap, and completion scope determine the cost to useful work.

4.3.3 What does the NIC pay? Sending and serving many connections. RC connections keep per-connection QP state in the NIC’s Interconnect Context Memory (ICM), backed by host memory and cached on the NIC, alongside completion and memory-translation state. How does message rate change as the active connection set grows, does it matter whether the NIC sends, receives, or both, and can connection reuse or DC reduce the cost?

Setup. On P-H100, GPU threads issue RDMA writes through GPU-resident queues while a host handler rings the doorbells. We select active QP subsets from preallocated pools at a fixed GPU grid. Eight PEs on eight nodes exercise three roles: one sender, a receive-only target, or every NIC sending and receiving. Unless varied, each warp posts 32 writes per connection visit. We measure throughput including the final completion drain, vary payload and connection reuse, and corroborate the results with 32–128-PE all-to-all sweeps and host verbs.

Sending and receiving together drives the loss. A send-only NIC sustains its rate through much larger active sets, staying near 242 M msg/s as the set of active QPs grows to 32,768. When every NIC also receives, rate falls from 152 to 74 M msg/s over the same range (Figure 10a). Simultaneous traffic pays an initial penalty and loses a further 51% as the active set grows. Receive-only traffic sits between the two, declining only at about ten times larger active sets, and sooner with more senders. A connection count that is inexpensive for one traffic role or workload can be costly for another.

Connection reuse and payload change the outcome. Holding each connection for 8,192 writes per warp visit flattens the simultaneous-traffic curve to 139–143 M msg/s across 1,024–4,096 QPs (Figure 10a). Reuse removes much of the growing-set penalty, although the cost of simultaneous traffic remains. Conversely, changing connection on every write also hurts send-only traffic at large sets. Larger payloads hide the message-rate loss: at 4,096 QPs, simultaneous traffic retains only 55% of send-only goodput with 128 B writes but 98% from 256 B onward (Appendix D.3).

All-to-all loses 59% by 3,000 active connections. The dense 128-PE RC sweep places the fitted onset near 1,350 active connections per NIC. In the 32-PE sweep, cycling 31 peers loses 59% of the two-peer rate by roughly 3,000 connections (Figure 10b).

DC does not eliminate the loss. With 256 writes per destination, DC retains 95% of its two-peer rate around 1,984 DCI-peer pairs, but 23% at 2,976; RC with the same bursts retains 56% there (Figure 10b). Varying DCTs per PE from 1 to 8 does not remove this decline.

DC itself is cheap per operation, 0.45 μ s above RC through completion in mini-gda, but switching destination on every WQE costs NVSHMEM’s DC path 60 $\times$ in message rate on P-H100, most of which short per-destination bursts recover. The decline itself persists with host-resident queues, other queue depths, and host verbs without a GPU, and packet counters show no retransmission-driven amplification but cannot separate context-cache misses, packet-processing contention, and backpressure (Appendix D.3).

Takeaway. Traffic role, connection reuse, and payload determine the cost of active connections. With one PE per NIC, cycling 16 QPs across 127 peers already exceeds the all-to-all onset, even though a send-only NIC can sustain much larger sets. Queue count alone is therefore insufficient to budget NIC capacity.

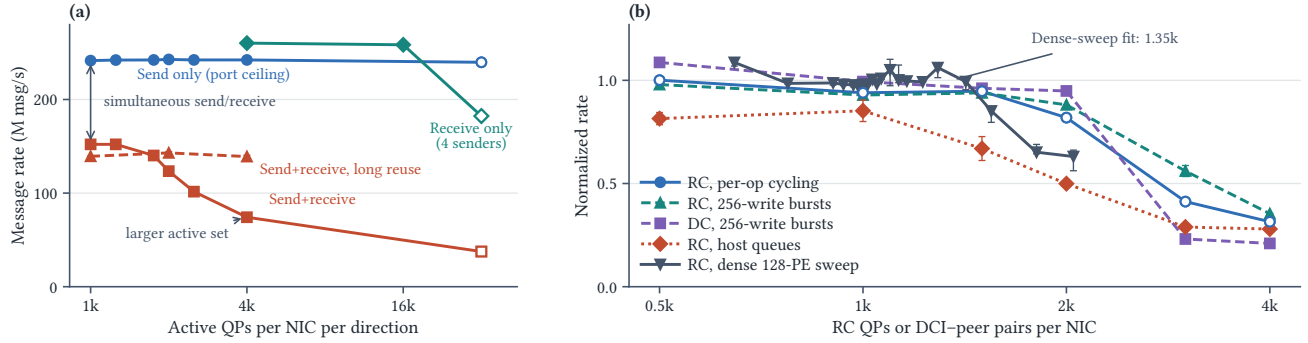


Figure 10: Connection scaling on P-H100. (a) 8 B rate through one NIC against its active QPs per direction. Long reuse increases writes per connection visit from 32 to 8,192. Receive-only uses total incoming QPs and aggregate rate over a common duration; other curves use the median sender. (b) 32-PE RC/DC and dense 128-PE RC sweeps, normalized to their two-peer controls within each pass. The x axis counts visited QPs or DCI-peer pairs. Medians over passes with their range; hollow markers are single-pass points; (b) uses the mean over NICs.

5 Related Work

GPU Communication Studies. Several recent works explore GPU communication stacks. Demystifying NCCL [16] analyzes NCCL’s protocols and algorithms, and Demystifying NVSHMEM [30] analyzes symmetric memory, transport selection, device-side RMA, collective algorithms, and DeepEP integration. The NCCL GIN paper [13] describes its device API and its GPU and proxy backends, focusing on the library rather than the mechanism beneath it. The Landscape of GPU-Centric Communication [51] surveys the field and establishes the initiation-vs.-submission taxonomy we adopt. Shen et al. [47] take the same mechanism-first approach inside an NVLink domain, deriving a speed-of-light bound from fence and remote-store latency and showing that synchronization, rather than data movement, dominates small-message collectives. Our work complements these studies by analyzing communication at the mechanism level through minimal GPU-submitted and proxy transports that vary WQE construction, ordering, queue sharing, and completion independently.

GPU-Initiated Communication in HPC. Agostini et al. [1] explore GPUDirect Async, where GPUs trigger InfiniBand communication that the CPU prepares, and Hamidouche and LeBeane [14] build a GPU-initiated OpenSHMEM that handles the GPU-NIC consistency problem of long-running kernels. Ismayilov et al. [19] propose a CPU-free execution model for iterative solvers, and Trotter et al. [49] compare CPU- and GPU-initiated communication for conjugate gradient solvers on GPU clusters. Baydamirli et al. [7] extend this model with compiler support that automatically transforms multi-GPU code to run without CPU orchestration. These studies evaluate communication strategies within applications; we measure the mechanisms those strategies rely on.

RDMA Performance Engineering. Kalia et al. [20] established several mechanisms as host-side optimizations, including doorbell batching, compact WQEs, payload inlining, and unsignaled completions, some of which have been adapted into GPU-side RDMA that we analyze in this work. Kong et al. [21] characterize NIC-side resources including QP context, address translation, and WQE cache. They find that large QP counts can pressure NIC-side caches and

degrade scaling, which motivates our active-connection experiment (§4.3.3) for GPU-driven traffic. FaRM shares connections among threads to limit this per-connection state [11]. ScaleRPC finds that inbound and outbound WRITES scale differently and groups connections for locality [8], and Collie reports bidirectional RDMA degradation from internal packet-processing contention [22]; our experiment separates traffic roles on ConnectX-7 and adds connection reuse and payload as variables.

Expert-Parallelism Libraries. DeepEP [53] introduced IBGDA-based MoE communication and remains the reference implementation, and pplx-kernels [24, 25] implements dispatch and combine over NVSHMEM. Hybrid-EP [52], NIXL [36], and NCCL EP [12] bring these designs into NVIDIA’s Megatron, Dynamo, and NCCL ecosystems, respectively. In contrast, fabric-lib [26], UCCL-EP [31], and NCCLX [48] route GPU requests through host threads for portability across GPU and NIC architectures. These libraries are the consumers of the mechanisms this paper dissects, and our measurements explain several of the differences in their published performance.

Beyond NVIDIA GPUs and ConnectX NICs. GPU-initiated communication is not exclusive to NVIDIA GPUs and ConnectX NICs. AMD’s rocSHMEM has GPU-submitting (GPUDirect Async) backends for ConnectX-7, Broadcom Thor 2, and Pensando Pollara NICs [5]. These backends, like IBGDA and GDAKI, submit from GPU code rather than only letting the NIC reach GPU buffers by DMA. The ROCm port of DeepEP drives ConnectX NICs from AMD GPUs through the same m1x5 doorbell path [6], and the MORI interface provides modular RDMA primitives for MI-series GPUs [4]. Intel SHMEM provides a device-initiated OpenSHMEM API for Intel GPUs [18].

On the NIC side, AWS EFA DP Direct posts work requests from CUDA, updating a 32-bit producer-index doorbell, rather than m1x5’s control-segment store, between system-scope fences [2, 3], and HPE Slingshot lets a GPU trigger NIC commands that the host prepared [15]. The mechanisms we measure (WQE construction, ordered doorbells, completion scope, and NIC connection state) apply to all of these paths, though their costs differ.

6 Conclusion

This paper dissected GPU-initiated communication at the GPU–NIC boundary. Using two minimal transports, mini-gda and mini-proxy, we separated the costs of the hardware mechanism from those of the libraries built on it, and measured both on four NVIDIA platforms. Our experiments connect queue management, ordering, and completion choices to their costs: the mechanism itself is cheap, and the choices a library makes around it determine latency, message rate, and the resources the kernel and the NIC must provide. We close with guidelines for library designers and for those who benchmark them.

- **Price the completion needed.** Choose completion scope independently of the queues needed for throughput. A PE-wide quiet walks every configured queue, while a signal or per-QP completion stays flat, so compare paths at the same queue count.
- **Report the processor state.** GPU submission depends on SM clock rate, while proxies depend on host operating state; report both for fair benchmarks and reproducibility.
- **Provision queues for isolation and capacity.** Reserve queues for latency-sensitive traffic in either design, and use batching and parallel submitters to raise capacity. A separate proxy ring isolates traffic even when it shares a worker.
- **Compile for the final caller.** Compile and measure communication in its final caller, where even dormant code can reduce useful throughput. Inspect register and stack usage for the actual kernel and block size, since neither inlining nor separate compilation is a universal default.
- **Budget connections by traffic, not by count.** Budget active connections by traffic role, reuse, and payload, which determine their cost to the NIC. DC reduces the number of persistent queues but does not remove this cost.

Understanding and using GPU-initiated communication efficiently demands expert knowledge and extensive testing, even inside vendor libraries. By dissecting the GPU–NIC boundary and measuring each mechanism in isolation, this paper aims to give users and developers of communication libraries an explainable reference point and a performance oracle for the transport. We release mini-gda, mini-proxy, and the benchmark suite as open source at <https://github.com/ParCoreLab/Dissecting-GPU-Communication-Experiments>.

Acknowledgments

Authors from Koç University have received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme (grant agreement No 949587). We acknowledge the EuroHPC Joint Undertaking for awarding access to the MareNostrum5 supercomputer in Spain, and TÜBİTAK ULAKBİM, High Performance and Grid Computing Center (TRUBA resources), where the experiments reported in this paper were partially performed.

References

- [1] Elena Agostini, Davide Rossetti, and Sreeram Potluri. 2018. GPUDirect Async: Exploring GPU synchronous communication techniques for InfiniBand clusters. *J. Parallel and Distrib. Comput.* 114 (2018), 28–45. doi:10.1016/j.jpdc.2017.12.007
- [2] Amazon Web Services. 2025. EFA DP Direct: GPU-initiated data path for Elastic Fabric Adapter. <https://github.com/amzn/efa-dp-direct> Accessed 2026-09-13.
- [3] Amazon Web Services. 2026. EFA DP Direct CUDA posting and completion implementation. https://github.com/amzn/efa-dp-direct/blob/5b50aab8fa0c81957cfe85461adc2e1201c8016f/CUDA/device/efa_cuda_dp_impl.cuh Accessed 2026-09-13.
- [4] AMD. 2026. MORI: Modular RDMA Interface. <https://github.com/ROCm/mori> Accessed 2026-09-13.
- [5] AMD. 2026. rocSHMEM 3.5.0 environment variables: GDA providers. https://rocm.docs.amd.com/projects/rocSHMEM/en/docs/7.14.0/env_variables.html ROCm 7.14.0 documentation. Accessed 2026-09-13.
- [6] AMD ROCm. 2025. DeepEP: A High-Performance Expert-Parallel Communication Library (ROCm port). <https://github.com/ROCm/DeepEP> Accessed 2026-09-13.
- [7] Javid Baydamirli, Tal Ben-Nun, and Didem Unat. 2024. Autonomous Execution for Multi-GPU Systems: Compiler Support. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, GA, USA). IEEE, 1129–1140. doi:10.1109/SCW63240.2024.00155
- [8] Youmin Chen, Youyou Lu, and Jiwei Shu. 2019. Scalable RDMA RPC on Reliable Connection with Efficient Resource Sharing. In *Proceedings of the Fourteenth EuroSys Conference 2019*. Association for Computing Machinery, New York, NY, USA, 14 pages. doi:10.1145/3302424.3303968
- [9] DeepSeek-AI. 2024. DeepSeek-V3 model configuration. <https://huggingface.co/deepseek-ai/DeepSeek-V3/blob/main/config.json> Hidden dimension 7,168. Accessed 2026-09-13.
- [10] DeepSeek-AI. 2026. DeepEP legacy IBGDA device path. https://github.com/deepseek-ai/DeepEP/blob/01dc3aaac82068020353dce2c302e38153c0bfaa/csrc/kernels/legacy/ibgda_device.cuh Single-QP quiet and exclusive-use requirement. Accessed 2026-09-13.
- [11] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. 2014. FaRM: Fast Remote Memory. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. USENIX Association, Seattle, WA, 401–414. <https://www.usenix.org/system/files/conference/nsdi14/nsdi14-paper-dragojevic.pdf>
- [12] Amos Goldman, Nimrod Boker, Maayan Sheraizin, Nimrod Admoni, Artem Polyakov, Subhadeep Bhattacharya, Fan Yu, Kai Sun, Georgios Theodorakis, Hsin-Chun Yin, Peter-Jan Gootzen, Aamir Shafi, Assaf Ravid, Salvatore Di Girolamo, James Dinan, Xiaofan Li, Manjunath Gorentla Venkata, and Gil Bloch. 2026. NCCL EP: Towards a Unified Expert Parallel Communication API for NCCL. arXiv:2603.13606 [cs.DC] <https://arxiv.org/abs/2603.13606>
- [13] Khaled Hamidouche, John Bachan, Pak Markthub, Peter-Jan Gootzen, Elena Agostini, Sylvain Jeaugey, Aamir Shafi, Georgios Theodorakis, and Manjunath Gorentla Venkata. 2025. GPU-Initiated Networking for NCCL. arXiv:2511.15076 [cs.DC] <https://arxiv.org/abs/2511.15076>
- [14] Khaled Hamidouche and Michael LeBeane. 2020. GPU Initiated OpenSHMEM: Correct and Efficient Intra-Kernel Networking for dGPUs. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP ’20)*. Association for Computing Machinery, New York, NY, USA, 336–347. doi:10.1145/3332466.3374544
- [15] Hewlett Packard Enterprise. 2025. HPE Cray MPI: GPU-NIC Async communication strategies. https://h41374.www4.hpe.com/docs/25.09/mpt/mpich/intro_mpi.html HPE Cray Programming Environment 25.09. Accessed 2026-09-13.
- [16] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff R. Hammond, and Torsten Hoefler. 2025. Demystifying NCCL: An In-Depth Analysis of GPU Communication Protocols and Algorithms. In *2025 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, San Jose, CA, USA, 48–59. doi:10.1109/HOTI66940.2025.00024
- [17] Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, Caio Rocha, Qinghua Zhou, Mahdieh Ghazimirsaeed, Sreevatsa Anantharamu, and Jithin Jose. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’26)*. Association for Computing Machinery, New York, NY, USA, 1201–1215. doi:10.1145/3779212.3790188
- [18] Intel. 2026. Intel SHMEM. <https://github.com/oneapi-src/ishmem> Accessed 2026-07-20.
- [19] Ismayil Ismayilov, Javid Baydamirli, Doğan Sağbılı, Mohamed Wahib, and Didem Unat. 2023. Multi-GPU Communication Schemes for Iterative Solvers: When CPUs are Not in Charge. In *Proceedings of the 37th ACM International Conference on Supercomputing (Orlando, FL, USA) (ICS ’23)*. Association for Computing Machinery, New York, NY, USA, 192–202. doi:10.1145/3577193.3593713

- [20] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. Design Guidelines for High Performance RDMA Systems. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*. USENIX Association, Denver, CO, 437–450. <https://www.usenix.org/conference/atc16/technical-sessions/presentation/kalia>
- [21] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R. Lebeck, and Danyang Zhuo. 2023. Understanding RDMA Microarchitecture Resources for Performance Isolation. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 31–48. <https://www.usenix.org/conference/nsdi23/presentation/kong>
- [22] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. 2022. Collie: Finding Performance Anomalies in RDMA Subsystems. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 287–305. <https://www.usenix.org/conference/nsdi22/presentation/kong>
- [23] Akhil Langer, Seth Howell, Aditya Goel, Pak Markthub, Heath Petty, and Fred Oh. 2024. Enhancing Application Portability and Compatibility Across New Platforms Using NVIDIA Magnum IO NVSHMEM 3.0. <https://developer.nvidia.com/blog/enhancing-application-portability-and-compatibility-across-new-platforms-using-nvidia-magnum-io-nvshmem-3-0/> Accessed 2026-07-20.
- [24] Nandor Licker, Kevin Hu, Vladimir Zaytsev, and Lequn Chen. 2025. Efficient and Portable Mixture-of-Experts Communication. <https://research.perplexity.ai/articles/efficient-and-portable-mixture-of-experts-communication> Accessed 2026-07-20.
- [25] Nandor Licker, Kevin Hu, Vladimir Zaytsev, and Lequn Chen. 2025. pplx-kernels: Perplexity MoE Kernels. <https://github.com/perplexityai/pplx-kernels>.
- [26] Nandor Licker, Kevin Hu, Vladimir Zaytsev, and Lequn Chen. 2026. fabriclib: RDMA Point-to-Point Communication for LLM Systems. In *Proceedings of Machine Learning and Systems*, Vol. 8. MLSys, Bellevue, WA, 169–185. https://proceedings.mlsys.org/paper_files/paper/2026/hash/dea9b4b6f55ae611c54065d6fc750755-Abstract-Conference.html
- [27] Linux RDMA Community. 2006. ibv_post_send(3): Work requests and completion signaling. https://man7.org/linux/man-pages/man3/ibv_post_send.3.html Upstream libverbs manual. Accessed 2026-09-13.
- [28] Linux RDMA Community. 2025. rdma-core mlx5 send posting and BlueFlame selection. <https://github.com/linux-rdma/rdma-core/blob/558104fc33266abe8a9deb50a81769d1a72fbf72/providers/mlx5/qp.c> Release v61.0, function post_send_db. Accessed 2026-09-13.
- [29] Linux RDMA Community. 2026. RDMA Core Userspace Libraries and Daemons. <https://github.com/linux-rdma/rdma-core> Includes libverbs and mlx5dv. Accessed 2026-09-13.
- [30] Yijun Ma, Siyuan Shen, Tiancheng Chen, Akhil Langer, Jiri Kraus, Benjamin Glick, Craig Belusar, Jeff Hammond, and Torsten Hoefer. 2026. Demystifying NVSHMEM: A System-Level Analysis on Symmetric Memory and Device-Initiated Operations in GPU Communication. arXiv:2606.05951 [cs.DC] <https://arxiv.org/abs/2606.05951>
- [31] Ziming Mao, Yihan Zhang, Chihan Cui, Zhen Huang, Kaichao You, Zhongjie Chen, Zhiying Xu, Zhenyu Gu, Scott Shenker, Costin Raiciu, Yang Zhou, and Ion Stoica. 2026. UEP: Portable Expert-Parallel Communication. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1107–1123. <https://www.usenix.org/conference/osdi26/presentation/mao-ziming-uep> Preprint titled “UCCL-EP: Portable Expert-Parallel Communication”, arXiv:2512.19849.
- [32] Pak Markthub, Jim Dinan, Sreeram Potluri, and Seth Howell. 2022. Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/> Accessed 2026-07-20.
- [33] NVIDIA. 2020. Mellanox Adapters Programmer’s Reference Manual (PRM). <https://network.nvidia.com/files/doc-2020/ethernet-adapters-programming-manual.pdf> Accessed 2026-09-13.
- [34] NVIDIA. 2022. NVIDIA OpenSHMEM Library (NVSHMEM) version 2.6.0 documentation. <https://docs.nvidia.com/nvshmem/archives/nvshmem-260/api/docs/introduction.html> Accessed 2026-07-18.
- [35] NVIDIA. 2025. GPUDirect RDMA: Synchronization and Memory Ordering. <https://docs.nvidia.com/cuda/archive/13.0.0/gpudirect-rdma/index.html#synchronization-and-memory-ordering> CUDA 13.0 documentation. Accessed 2026-09-13.
- [36] NVIDIA. 2025. NIXL: NVIDIA Inference Xfer Library. <https://github.com/ai-dynamo/nixl> Accessed 2026-07-20.
- [37] NVIDIA. 2026. DOCA GPUNetIO device queue posting, bundled with NCCL 2.31.2. https://github.com/NVIDIA/nccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/src/transport/net_ib/gdaki/doca-gpunetio/include/device/doca_gpunetio_dev_verbs_qp.cuh Release v2.31.2-1. Accessed 2026-09-13.
- [38] NVIDIA. 2026. Dynamically Connected QPs. <https://networking-docs.nvidia.com/doca/archive/3-5-0/dynamically-connected-qps> DOCA 3.5.0 documentation. Accessed 2026-09-13.
- [39] NVIDIA. 2026. GDRCopy: A low-latency GPU memory copy library based on NVIDIA GPUDirect RDMA. <https://github.com/NVIDIA/gdrcopy> Accessed 2026-09-13.
- [40] NVIDIA. 2026. NCCL 2.31.2 Device API: GIN. https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/api/device_gin.html Context, flush, flushAsync, and wait contracts. Accessed 2026-09-13.
- [41] NVIDIA. 2026. NCCL 2.31.2 GDAKI device implementation. https://github.com/NVIDIA/nccl/blob/7b83616df3ae082a1f32bb74c27458bfe8153a13/src/include/ncl_device/gin/gdaki/gin_gdaki.h Release v2.31.2-1. Accessed 2026-09-13.
- [42] NVIDIA. 2026. NVIDIA OpenSHMEM Library (NVSHMEM) documentation. <https://docs.nvidia.com/nvshmem/api/index.html> Accessed 2026-09-13.
- [43] NVIDIA. 2026. NVSHMEM 3.7.2 IBGDA device implementation. https://github.com/NVIDIA/nvshmem/blob/3f4c6d81225f45f9c42ed9af09bdbc61eb356d4/src/include/non_abi/device/pt-to-pt/ibgda_device.cuh Release v3.7.2-0. Publication, posting, CQ polling, and QP-specific quiet. Accessed 2026-09-13.
- [44] NVIDIA. 2026. NVSHMEM 3.7.2 IBGDA host setup and CPU-assisted progress. <https://github.com/NVIDIA/nvshmem/blob/3f4c6d81225f45f9c42ed9af09bdbc61eb356d4/src/modules/transport/ibgda/ibgda.cpp> Release v3.7.2-0. Accessed 2026-09-13.
- [45] NVIDIA. 2026. NVSHMEM 3.7.2 transport configuration defaults. https://github.com/NVIDIA/nvshmem/blob/3f4c6d81225f45f9c42ed9af09bdbc61eb356d4/src/modules/transport/common/env_defs.h Release v3.7.2-0. Accessed 2026-09-13.
- [46] Sreeram Potluri, Khaled Hamidouche, Akshay Venkatesh, Devendar Bureddy, and Dhableswar K. Panda. 2013. Efficient Inter-node MPI Communication Using GPUDirect RDMA for InfiniBand Clusters with NVIDIA GPUs. In *2013 42nd International Conference on Parallel Processing*. IEEE, Lyon, France, 80–89. doi:10.1109/ICPP.2013.17
- [47] Siyuan Shen, Anton Korzh, John Bachan, Tiancheng Chen, Arnav Goel, Ludwig Schneider, Pouya Kousha, Zhenhao He, Sylvain Jauegy, Kamil Iskra, Nishank Chandawala, Jeff R. Hammond, and Torsten Hoefer. 2026. Every Microsecond Matters: Achieving Near Speed-of-Light Latency in GPU Collectives. arXiv:2607.16100 [cs.DC] <https://arxiv.org/abs/2607.16100>
- [48] Min Si, Pavan Balaji, et al. 2025. Collective Communication for 100k+ GPUs. arXiv:2510.20171 [cs.DC] <https://arxiv.org/abs/2510.20171>
- [49] James D. Trotter, Sinan Ekmekci, Dogan Sagbili, Johannes Langguth, Xing Cai, and Didem Unat. 2025. CPU- and GPU-initiated Communication Strategies for Conjugate Gradient Methods on Large GPU Clusters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (St. Louis, MO, USA) (SC ’25)*. Association for Computing Machinery, New York, NY, USA, 298–315. doi:10.1145/3712285.3759774
- [50] Ultra Ethernet Consortium. 2025. Ultra Ethernet Specification v1.0.1. <https://ultraethernet.org/wp-content/uploads/sites/20/2025/10/UE-Specification-1.0.1.pdf> Accessed 2026-09-13.
- [51] Didem Unat, Ilyas Turimbetov, Mohammed Issa, Dogan Sagbili, Flavio Vella, Daniele De Sensi, and Ismayil Ismayilov. 2026. The Landscape of GPU-Centric Communication. *Comput. Surveys* 58, 12, Article 322 (Sept. 2026), 36 pages. doi:10.1145/3813799
- [52] Fan Yu, Tong Liu, and Kai Sun. 2026. Optimizing Communication for Mixture-of-Experts Training with Hybrid Expert Parallel. <https://developer.nvidia.com/blog/optimizing-communication-for-mixture-of-experts-training-with-hybrid-expert-parallel/> Accessed 2026-07-20.
- [53] Chenggang Zhao, Shangyan Zhou, Liye Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>. Accessed 2026-07-20.

A Methodology Details

As a timer check, adding a 50 μ s responder delay shifts round trips by 49.6–50.3 μ s on both P-IB and P-GB200. The SM clock measured 1,962–1,980 MHz during P-IB latency runs and 2,062 MHz on P-GB200. Runs discard 100–1,000 warm-up operations. Section 4.1 and the idle, handoff, message-rate, and payload measurements of Section 4.2 use one P-IB pair; the loaded-RTT curves, including their idle controls, come from a second pair.

B Single-Operation Controls

B.1 SM-clock sensitivity

The P-RoCE sweep uses NVSHMEM 3.7.2 with two RC QPs per peer and NCCL 2.31.2 GDAKI with one context. Three shuffled passes measure 5,000 operations per cell at six SM clocks (502–1,845 MHz); payload and NIC packet-count checks pass.

Table 6: Selected P-RoCE fits, $T(f) = C/f + B$, using six clock medians, with T in μ s and f in MHz. C is an effective clock-sensitive coefficient.

Sequence	C (cycles)	B (μ s)	R^2
NVSHMEM public issue	10,313	1.44	0.9970
NVSHMEM public put+completion	16,987	12.39	0.9963
GDAKI inline issue	2,929	0.27	0.9977
GDAKI inline put+flush	5,683	10.30	0.9432
IBRC enqueue	1,891	0.09	0.9986
IBRC put+completion	6,045	8.91	0.9514

B.2 Completion scope

The P-RoCE scope experiment uses three shuffled passes of 5,000 operations after 1,000 warm-up operations at a measured 1,965 MHz. Both used-QP routines complete the QP selected by the same internal put.

Table 7: P-RoCE completion scope, NVSHMEM 3.7.2: 8 B latency. All-QP, DeepEP *port*, and NVSHMEM *stock* completion use the same internal put. DeepEP and *stock* complete only its selected QP. RTT is a signal-based control.

QPs/peer	Put+completion (μ s)			RTT (μ s)
	All QPs	Used, port	Used, stock	
1	18.02	16.93	17.25	31.17
2	18.78	16.90	17.09	31.78
4	19.78	15.14	17.98	31.94
8	20.96	15.87	17.60	31.58
16	24.77	17.09	17.92	30.98
32	34.85	16.93	15.49	29.98

Table 8: P-GB200 NVSHMEM 3.7.2 put+completion. DCI requested = 0 selects automatic sizing. One GPU and NIC per node, all traffic through the NIC. SM clock at 2,062 MHz.

RC/peer	DCI requested	DCI actual	Put+completion (μ s)	
			Public	Internal
2	1	1	13.86	12.19
16	1	1	22.98	21.28
2	0	153	94.88	92.86

C Proxy Controls

C.1 Operating state

Cold workers follow 45 s idle (2.5 GHz); warm workers follow 20 s sustained load (4.0 GHz). CPU-only preconditioning reproduces the warm results while GPU-submitted controls change little.

Table 9: P-IB proxy latency, measured cold and warm in one session on one pair.

Path	Put+c. (μ s)		RTT (μ s)		RTT p99 (μ s)	
	Cold	Warm	Cold	Warm	Cold	Warm
IBRC	6.40	6.30	11.07	10.69	12.80	11.46
GIN Proxy	8.19	7.57	16.32	14.53	16.86	15.07
mini-proxy, baseline	7.07	6.91	14.59	13.44	16.93	13.86
mini-proxy, tuned	4.10	3.62	5.89	5.22	8.86	5.57

Table 10: Selected P-IB proxy capacity.

Path	Workers	M msg/s	
		Cold	Warm
mini-proxy, $B = 1$	1	2.6	6.6
mini-proxy, $B = 16$	1	14.4	28.6
UCCL-EP	8	34.0	60.8
MSCCL++, patched	8	7.5	10.2
GIN Proxy, 4 contexts	4	3.3	7.0
fabric-lib	1	8.6	12.6

MSCCL++ Patch. Passing memory handles by const reference removes shared-pointer refcount contention, raising P-IB capacity from 2.5 to 7.5 M msg/s over one to eight services. The patched build retains one signaled post per message and is used throughout the paper.

C.2 Queue isolation

Table 11: Loaded 8 B RTT p50, 64 background CTAs. GDAKI uses eight threads per CTA and a fixed 65-context pool: the shared probe uses one background CTA’s context. mini-proxy uses T4/R32/B16; its shared/private initiator rates are 47.5/44.9 M msg/s on P-IB and 61.8/62.9 on P-GB200. UCCL-EP uses eight workers. Paced rows compare reserved queues at approximately 30 M msg/s.

Path	RTT (μ s)	
	Shared	Reserved
<i>Unpaced</i>		
GDAKI, P-IB	27.3	13.9
mini-proxy, P-IB	2,190	115
mini-proxy, P-GB200	1,401	178
UCCL-EP, P-IB	7,570	120
MSCCL++, P-IB	50,800	825
<i>Paced to about 30 M msg/s</i>		
GDAKI, P-IB	—	12.9
mini-proxy, P-IB	—	29.2

C.3 Payload size

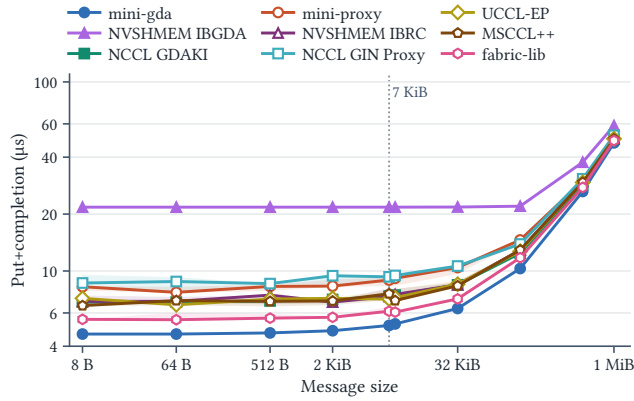


Figure 11: Single-operation put+completion against payload size on P-IB, for the paths of Figure 8. NVSHMEM uses all-QP quiet.

Table 12: Selected P-GB200 pointer-write goodput, with four CTAs at 4 KiB and one at 64 KiB. mini-proxy uses T4/R32/B16, IBGDA 16 RC QPs, GIN Proxy four requested workers and one context per CTA, and GDAKI 32 threads and one context per CTA. Coarse sizes and changing geometry do not locate a crossover.

Path	Goodput (GB/s)	
	4 KiB	64 KiB
mini-proxy	49.1	49.0
NVSHMEM IBGDA	49.3	49.5
GIN Proxy	47.9	48.5
GDAKI	21.3	48.0

D Resource and Scaling Controls

D.1 Queue publication

Lane drift also explains a warm-up sensitivity on P-GB200: lengthening the warm-up from 100 to 1,000 iterations cuts GDAKI’s aggregated rate from 67 to 17 M msg/s, while synchronizing after each put keeps it at 82 M msg/s in both cases (Table 13).

Table 13: Selected 8 B message-rate controls. P-IB GDAKI uses 16 contexts, one per CTA; synchronizing after each put raises the 32-thread case to 106–110 M msg/s. NVSHMEM and mini-gda use 256 threads per CTA and one QP per CTA; mini-gda publishes cooperatively with 32 WQEs per doorbell. P-GB200 GDAKI uses 16 contexts of 32 threads, aggregates every 16 puts, disables NCCL’s RAS monitoring, and measures 10,000 iterations.

P-IB configuration	M msg/s	
GDAKI, 1 / 8 threads per context	12.0 / 39.5	
GDAKI, 32 / 64 threads per context	34.3 / 27.2	
NVSHMEM, 2 / 16 RC QPs	50.2 / 251.5	
mini-gda, 16 QPs / 4,096 threads	260	
P-GB200 warm-up iterations	Original	Sync after put
100	67.10	82.20
1,000	16.61	82.12

D.2 Enclosing-kernel resources

Kernel times are medians of three paired passes of 15 launches. The inlined build sets NVSHMEM_ENABLE_ALL_DEVICE_INLINING=ON.

Table 14: P-IB compute caller with eight live values (sm_90, -03). Waiting loss is relative to sending without waiting, with 16 individual 8 B messages per block.

Path	Registers/thread		
	Absent	Dormant	Waiting loss (%)
mini-gda	29	32	1.9
mini-proxy	29	34	26.8
NVSHMEM, separate	34	44	22.3
NVSHMEM, inlined	34	98	18.8
GDAKI	29	95	5.3

With dormant NVSHMEM code and the earlier instrumented builds, a compute caller with 160 live values retains 15% of its throughput when NVSHMEM is compiled separately and 100% when it is inlined; at 224 live values, it retains 8% and 25%.

DeepEP Configuration. DeepEP V1 (NVSHMEM 3.4.5) uses two four-GPU P-IB nodes, 128 tokens per rank, hidden size 7,168, 288 experts, top-8 routing, and all-RDMA traffic. Across five runs, combine finishes issuing after 24 μ s, and its median warp spends 0.93 ms at grid synchronization before reduction.

D.3 Active connections

The dense 128-PE sweep fits a plateau joined to a power-law decline by least squares in log space.

Table 15: P-H100 active-connection controls. GPU runs use CPU-forwarded doorbells and normalize to each configuration’s two-peer control. Host verbs normalize to their one-sender control.

Control	Observation
RC, host / GPU queues	0.50 / 0.82 at 1,984 connections; 0.29 / 0.41 at 2,976
Queue depth	Rate changes within $\pm 2\%$
DC, 1–8 DCTs per PE	0.95 at 1,984 DCI–peer pairs; 0.23–0.24 at 2,976
Host verbs, send + receive	0.60–0.75 at 1,024 QPs/direction; ≤ 0.10 from 2,048
Host verbs, one sender	Flat through 7,168 QPs
Packets, send + receive	1.01–1.09 per message, ≤ 0.1 ACK per received message over 1,024–7,168 QPs

Table 16: Selected P-H100 payload controls: 4,096 active QPs per NIC, four peers, 32 writes per connection visit. Goodput is the median sender rate per pass, then median [min, max] over passes (three at 8 B, two otherwise). Ratio divides the two medians.

Size (B)	Goodput (GB/s)		
	Send only	Send + receive	Ratio
8	1.94 [1.94, 1.94]	0.59 [0.58, 0.61]	0.31
128	18.12 [18.12, 18.13]	9.90 [9.71, 10.09]	0.55
256	21.00 [21.00, 21.00]	20.65 [20.64, 20.66]	0.98
4,096	24.68 [24.68, 24.68]	24.35 [24.35, 24.35]	0.99